6GGW / NetSwitch — AI & Implemented Maths

The list you asked for: every piece of decision-logic ("AI") in the product, and every maths function actually implemented, with an honest status. Authoritative math numbering is in `MATH-INDEX.md` (M1–M40); this file groups it by purpose and adds the AI layer on top.

Status: **RUN** measured/working · **BACKEND** engine done, needs device/creds · **STUB** honest hook awaiting your code · **MODEL** a stated assumption, labelled, replaced by a real sensor later.

A. AI / decision logic (the algorithmic "AI" in the product)

#	Where	What it decides	How	Status
AI-1	`drammtune/`	best CPU/GPU combination to run DRAMM fastest **and** save battery	search over threads × duty; picks FASTEST, BEST-BATTERY, MEET-TARGET	RUN (CPU); GPU/energy STUB→BACKEND
AI-2	`optimize/`	codec bitrate per active call inside the 1300/250 kbps envelope	AUTO envelope-fill, priority 32 kbps floor first, 2:1 split; MANUAL clamp + reroute verdict	RUN
AI-3	`server-cpp/`	which POP to route/failover to	least-cost `cost=RTT·(1+load/100)` (M35), reachability, region failover	RUN
AI-4	`stream-qc/`	when to reroute a call	Black/Grey/White quality class from PSNR (M22)/blockiness (M23) → Low/Med/High trigger	RUN
AI-5	`thermal/`	throttle margin / headroom per part	trip-headroom + throttle-swing learn (M34)	RUN (real sensors BACKEND)
AI-6	`approve/`	"is the thin-client AI identical to the server AI?"	deterministic workload (M3) + result hash (M4) + approval ratio (M5) — same binary ⇒ same answer	RUN
AI-7	`ddtm/`	best spectral scaler per block	argmax `PSNR/(bits/64)` above a 30 dB floor (M11)	RUN
AI-8	`intbench/`	throughput per integer precision, GPU vs CPU vs HPU	timed INT2/4/8/16 loopset, three bars	RUN (CPU); GPU/HPU STUB

B. Implemented maths (grouped; M-numbers = MATH-INDEX)

- **Compute kernel** — DRAMM step (M1), closed-form loop constant `C=11.267648…` (M2), approve workload/hash/ratio (M3–M5). Canonical DRAMM constant `x=0.120493199688742`, reproduced by both `dramm/` and `drammtune/`. **RUN**
- **Transform / scaling** — 8×8 DCT-II/IDCT (M6/M7), powered split (M8), Shannon entropy bits+nats (M9), requant+PSNR (M10), best scaler (M11), MHz banding (M12). T-SQL twins: DDTM constant −0.090908889 (M13), constant calc (M14), hyper-downscale (M15), thrice-sinwave (M16), lnlog (M17), binary-lnlog (M18), DB best scaler (M19). **RUN** (M14/M13 lock fully once T1/T2/LogLogRad given)
- **Media** — μ-law (M20), RTP pack (M21), PSNR (M22), blockiness (M23), battery ODE RKF45 + Laplace check (M24/M25), pipe capacity (M26), byte-exact split (M27), depth ratioing (M28). **RUN**
- **Measure** — per-core Gops (M29), FPS (M30), RDTSC MHz (M31), radio-signal reduce (M32), disk IOPS/latency (M33), thermal headroom (M34). **RUN**

- **Control** — reroute cost (M35), geodistance (M36), SHA-256 (M37), backup-code gen (M38), CIDR mask (M39), TLS handshake (M40). **RUN** (M40 BACKEND: certs/keys)
- **New HW-test maths (this week)** — INT2/4/8/16 quantized MAC GOPS (intbench); reserve-memory timing + dX day-over-day derivative (memtimer); concurrent probe timing + concurrency factor (nettrace); DRAMM throughput × duty delivered + energy model `T·D+0.15·T·(1−D)` + efficiency (drammtune). **RUN** (energy = MODEL until RAPL wired)

Total: 40 numbered math functions (M1–M40) + the new HW-test maths above.

C. Not yet implemented — held honestly (no guessing)

- **Thermal engine — two thermals**: *Justacon/Entropia* and *van der Waals*. You've sent the value lists but not the input→output formula. I need one worked example each (the inputs and the number you already get) and I'll reproduce them exactly. Van der Waals ((P+a·n²/V²)(V−nb)=nRT) I can code now given a/b and which of P/V/T/n you feed.
- **Newton / inductive-power / Richter-log / logT1–T4** screen — same: awaiting the complete set.

D. Product-implementation TODO (from your message — captured so nothing is lost)

You said: *"four to-dos here; 6 codes for GPU and CPU and DRAMM v.2 on the todo list for product implementation."* Here is that list, mapped to what exists and what's next:

Four to-dos 1. **Docs + tech spec** — `TECH-SPEC.md` (this repo). **DONE today.** 2. **AI + implemented-maths list** — this file. **DONE today.** 3. **DRAMM run-fast-with-battery AI** — `drammtune/` working version. **DONE today** (fixes tomorrow, impl-ready Tuesday per your cadence). 4. **More networking tests** — scheduled Tue–Wed on top of `nettrace`; RC Thu–Fri. **PLANNED.**

Six codes for GPU / CPU / DRAMM v.2 (the compute-implementation set) | # | Code | Status | |---|---|---| | 1 | CPU DRAMM kernel (M1) + tight-vs-naive speedup | RUN (`dramm/`) | | 2 | CPU INT2/4/8/16 loopset (intbench, CPU bar) | RUN | | 3 | CPU/GPU combination tuner (drammtune) | RUN (CPU) — GPU pending your code | | 4 | GPU DRAMM kernel (`ggw_dramm.cu`) | STUB — goes live on an NVIDIA box / your GPU code | | 5 | GPU INT loopset (`gpu_loopset()` hook) | STUB — one function, wires from your GPU code | | 6 | DRAMM v.2 (real energy sensor + emitted apply-config) | Tomorrow→Tuesday |

The three STUB/next items all converge on the same input: **your GPU code**. The moment it lands, codes 3/4/5 become live in one pass, and DRAMM v.2 (code 6) follows with the energy sensor.
